

Prof Xiaowei Huang

<https://cgi.csc.liv.ac.uk/~xiaowei/>

(Attendance Code: **567714**)

Lecture 20 -- Transformer



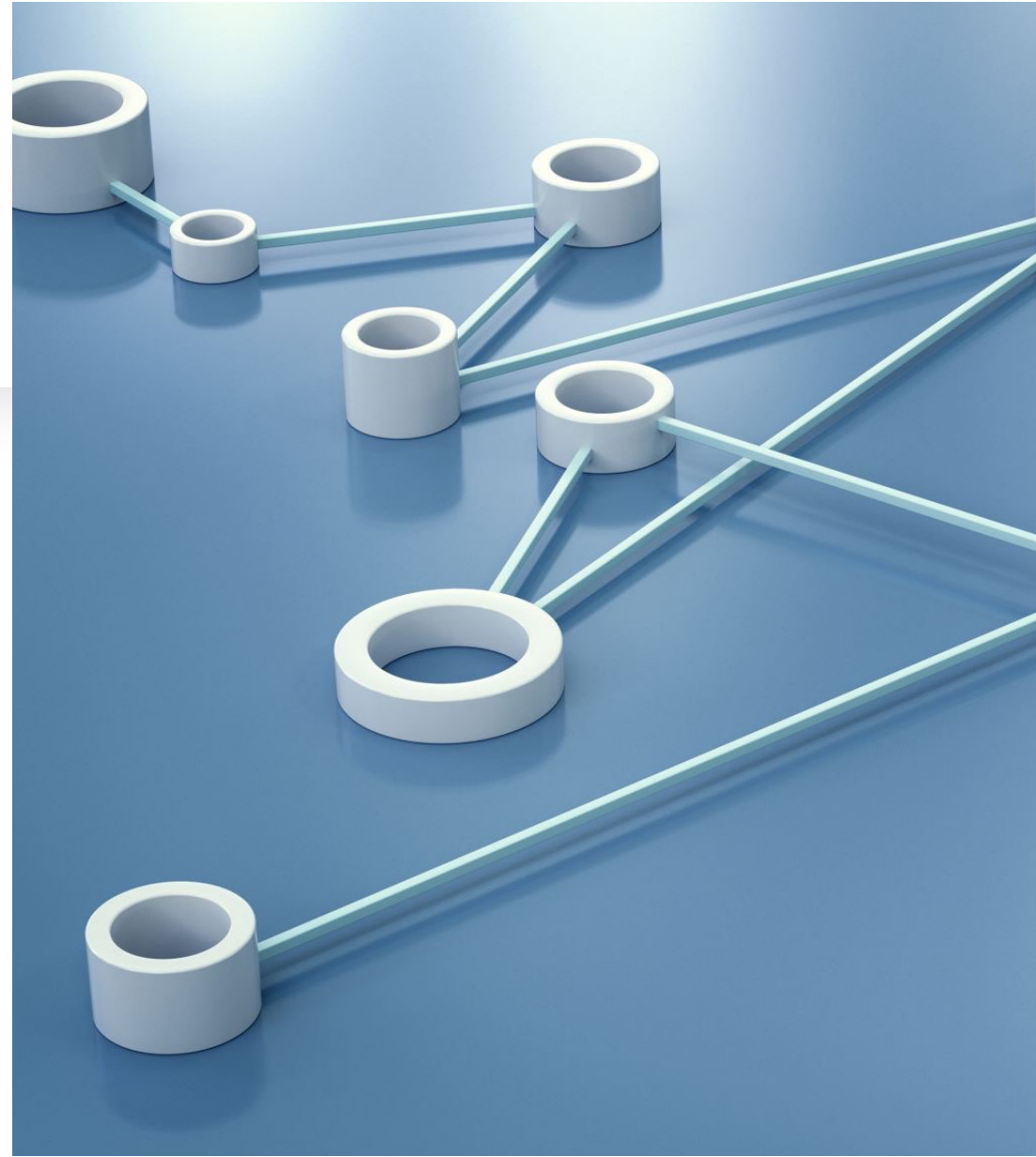


Up to now,

- Traditional Machine Learning Algorithms
- Adversarial Attack and Defence
- Deep learning
 - Introduction to Deep Learning
 - Functional view and features
 - Backward and forward computation (including backpropagation and chain rule)
 - Convolutional neural networks
 - Recurrent neural networks

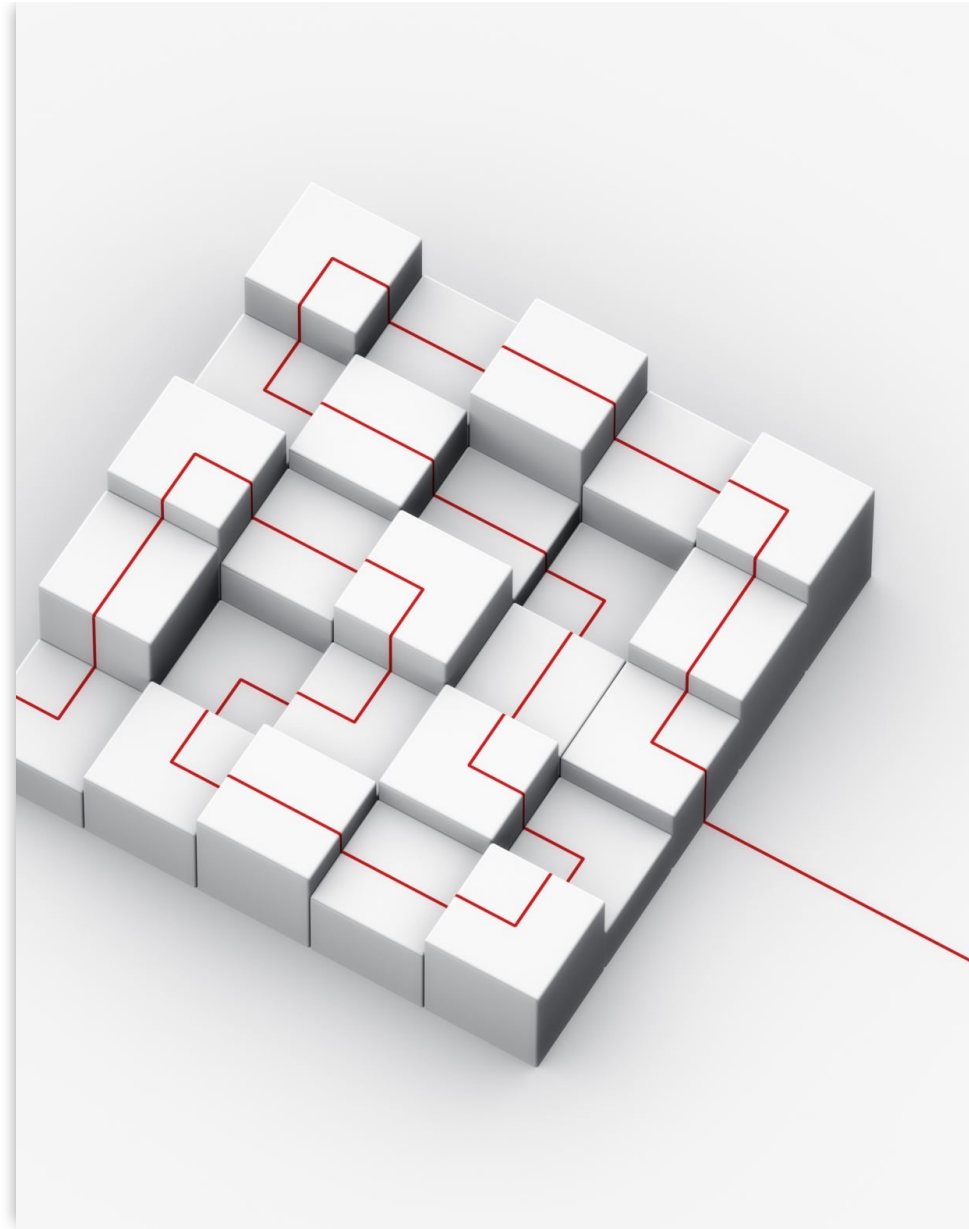
Topics

- General Pre-trained Transformer vs Large Language Models
- Transformer Architecture
 - Positional Encoding
 - Self-Attention
 - Multi-head attention
 - Encoder
 - Decoder
 - Masking
 - Next-word prediction



Transformer

- Developed by researchers at Google in 2017
- State-of-the-art natural language processing (NLP) model
- Basic building block of most large language models, including ChatGPT and GPT-4

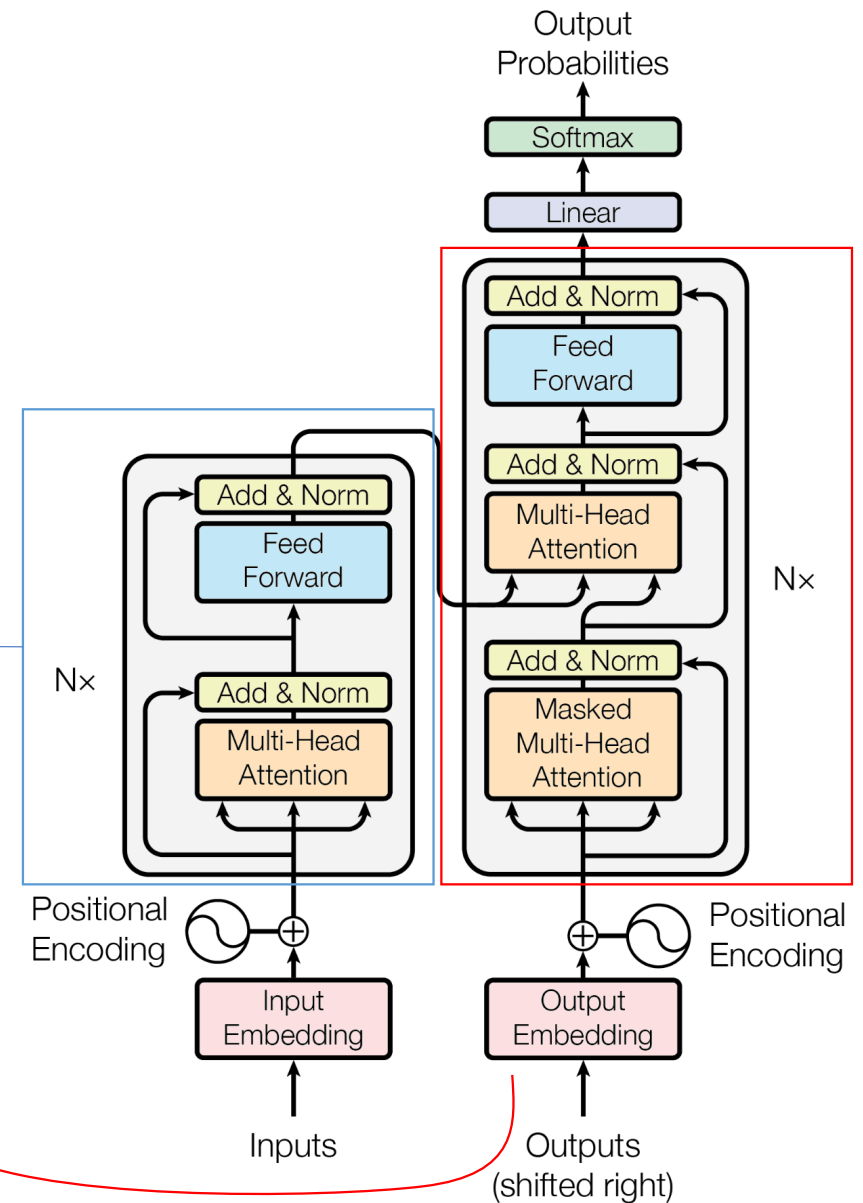


Transformer vs Large Language Models

- Large language models (LLMs) are heavily based on the transformer architecture
 - Examples of large language models include GPT-3, GPT-4, BERT, and RoBERTa.
 - Examples of general pre-trained transformers include ViT (Vision Transformer) for computer vision tasks and Conformer models for speech recognition tasks.
- LLMs are first pre-trained using the transformer architecture to learn general language patterns. Then, they can be fine-tuned on specific tasks, often with a relatively small dataset, leveraging the transformer's broad understanding of language.
- Transformers use a *self-attention mechanism*, allowing each word in a sequence to focus on other words in that sequence, no matter how far apart they are

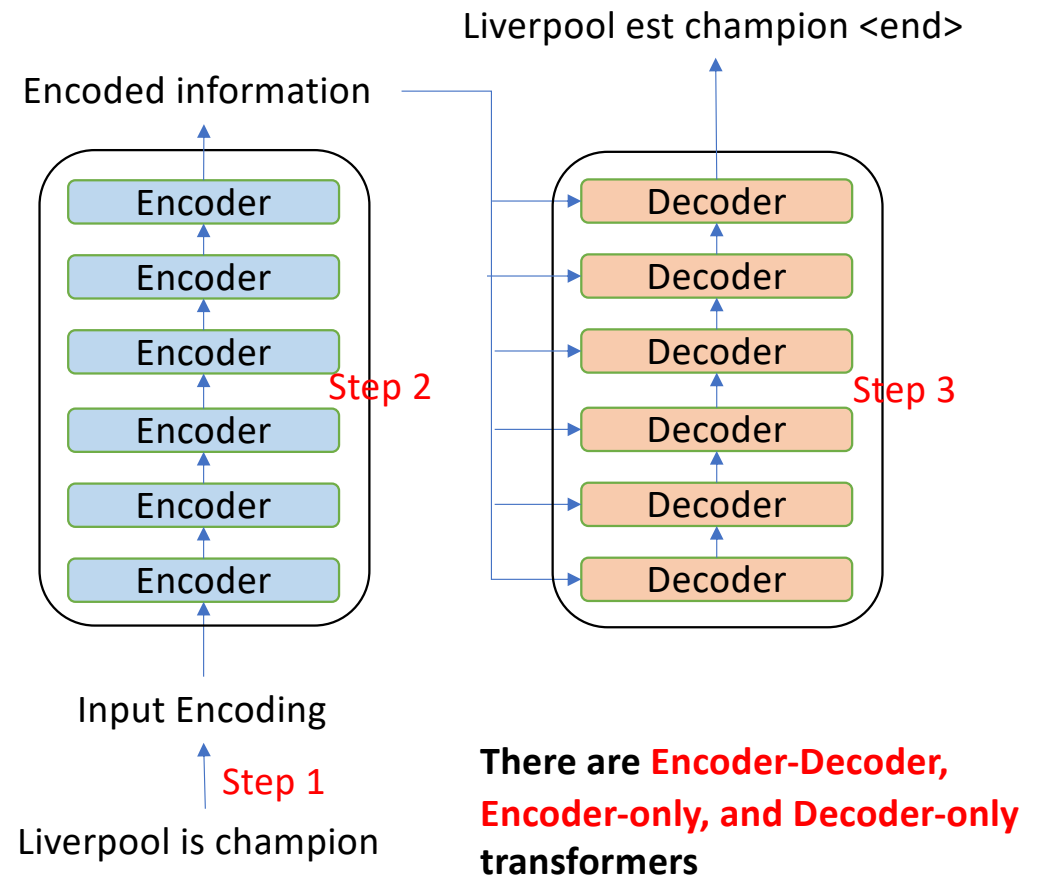
Transformer Introduction

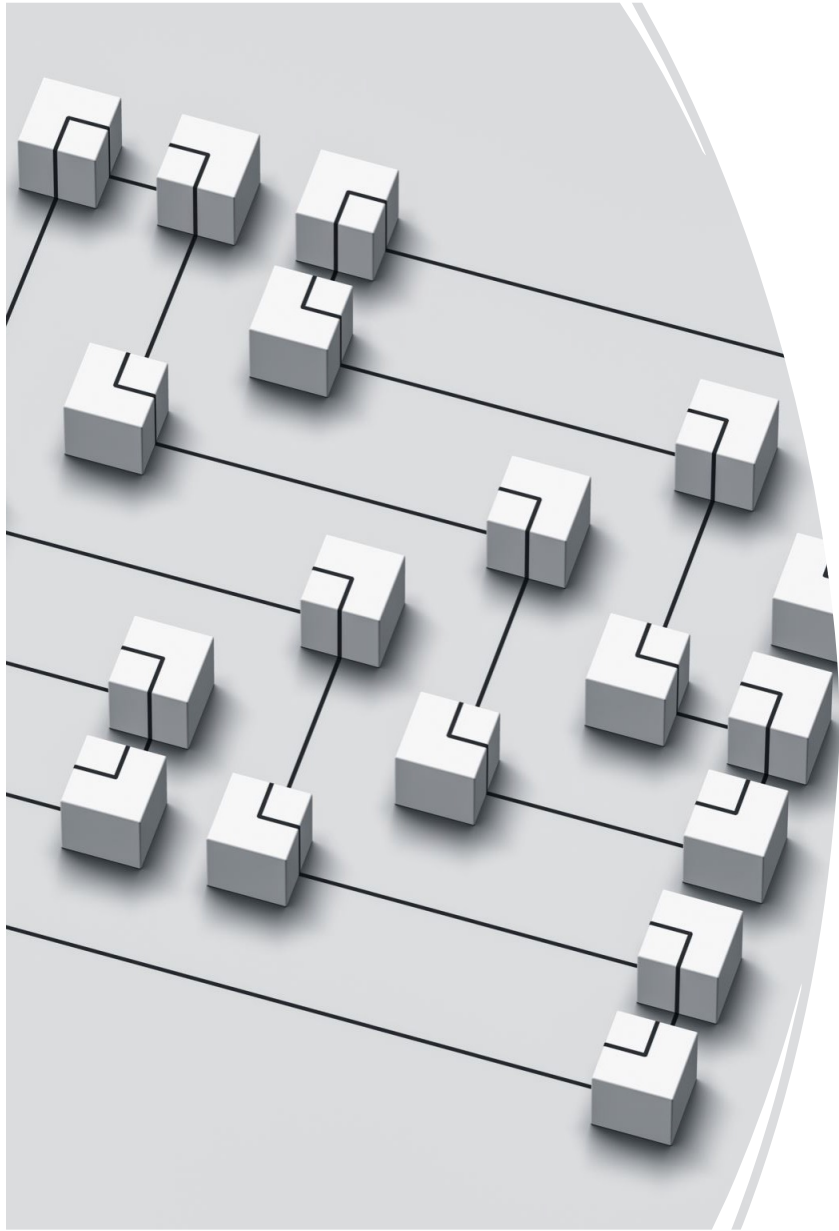
- The Transformer architecture follows an encoder-decoder structure but does not rely on recurrence and convolutions in order to generate an output.
- The task of the encoder, on the left half of the Transformer architecture, is to map an input sequence to a sequence of continuous representations, which is then fed into a decoder.
- The decoder, on the right half of the architecture, receives the output of the encoder together with the decoder output at the previous time step to generate an output sequence.
- At each step the model is auto-regressive, consuming the previously generated symbols as additional input when generating the next.



Transformer Unfolded

- English to French translation
- Example on the right:
 - 6 encoder blocks
 - 6 decoder blocks
- Three steps:
 - Input encoding
 - Encoder
 - decoder





Encoder-Decoder (Sequence-to-Sequence) Transformers

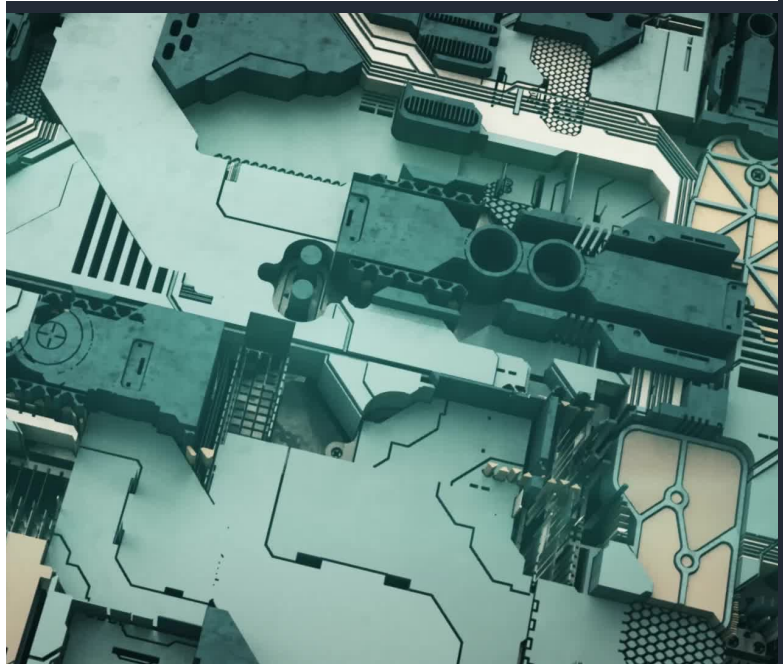
- The *encoder* processes the input sequence, transforming it into a hidden representation, while the *decoder* generates an output sequence based on this hidden representation.
- Notable encoder-decoder models include **T5 (Text-To-Text Transfer Transformer)** and **BART (Bidirectional and Auto-Regressive Transformers)**, commonly used for tasks like translation and summarization.



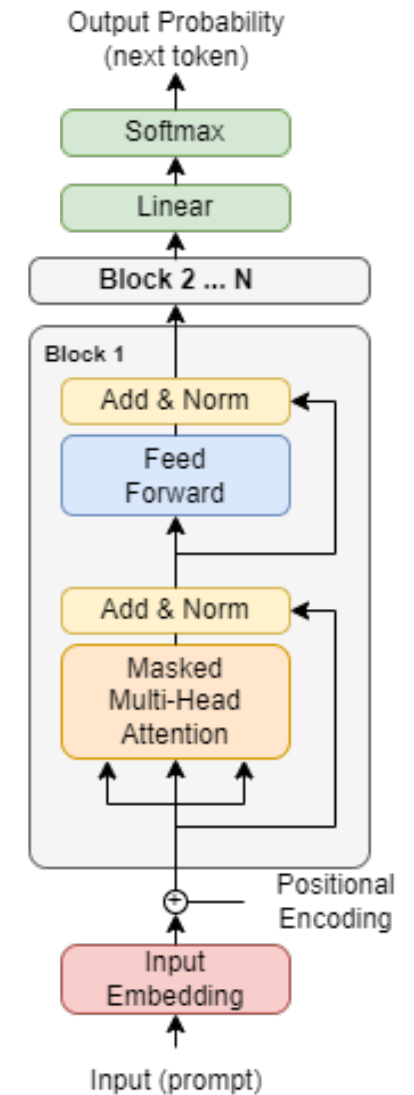
Encoder-Only Transformers

- Encoder-only transformers consist solely of encoder layers, which process the input sequence bidirectionally (i.e., looking at context in both directions).
- This architecture encodes the input sequence into a fixed-length representation but does not have a decoder component for autoregressive generation.
- **BERT (Bidirectional Encoder Representations from Transformers)** and **RoBERTa** are well-known encoder-only models designed primarily for tasks like classification, entity recognition, and sentiment analysis

Decoder-Only Transformers

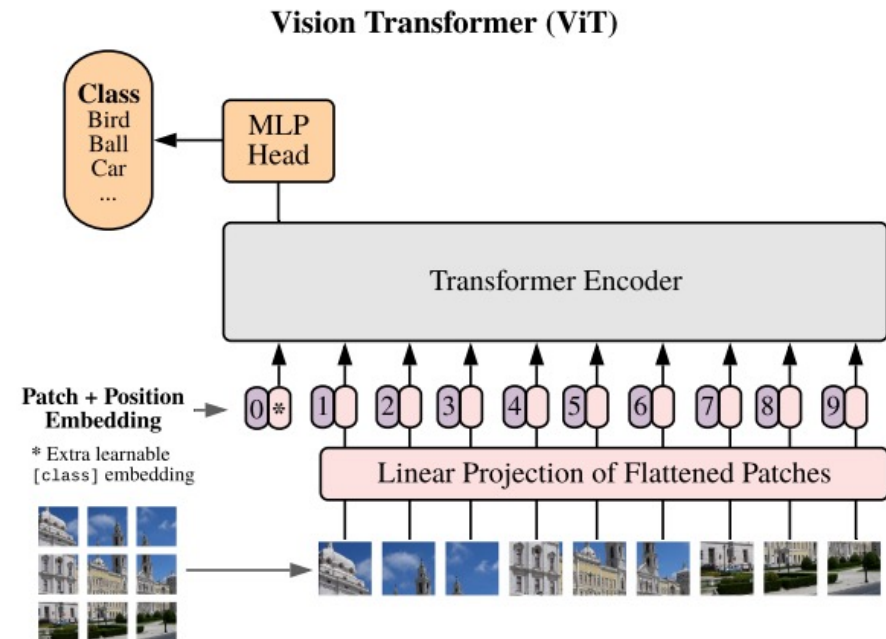


- Decoder-only transformers use only the decoder part of the transformer, which generates tokens sequentially in an autoregressive fashion (predicting the next token based on prior ones).
- These models can't view the full input sequence at once like the encoder-only transformers. Instead, they rely on causal (or autoregressive) self-attention to prevent "seeing" future tokens.
- Decoder-only models are optimized for *unidirectional generation* tasks, such as text generation, language modeling, and conversational response generation.
- **GPT (Generative Pre-trained Transformer)** series (like GPT-3 and GPT-4) are decoder-only models widely used for text generation, language modeling, and chatbots.



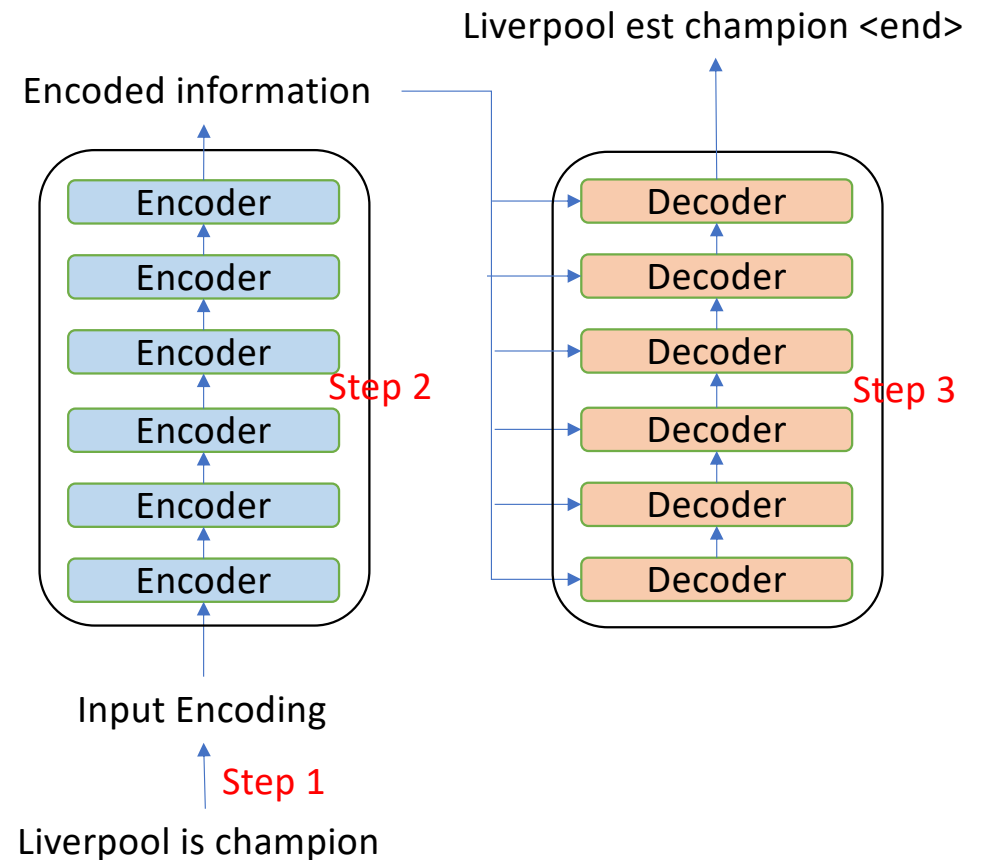
Vision Transformer

- CNNs scan small patches of the image to extract features like edges or textures, but what if we need to see the entire picture to get a good understanding?
- Vision Transformers (ViT) breaks images down into patches and treats each patch as if it were a “word” in a sentence. This allows it to capture relationships between different parts of the image, like how the ear of the cat relates to the tail.



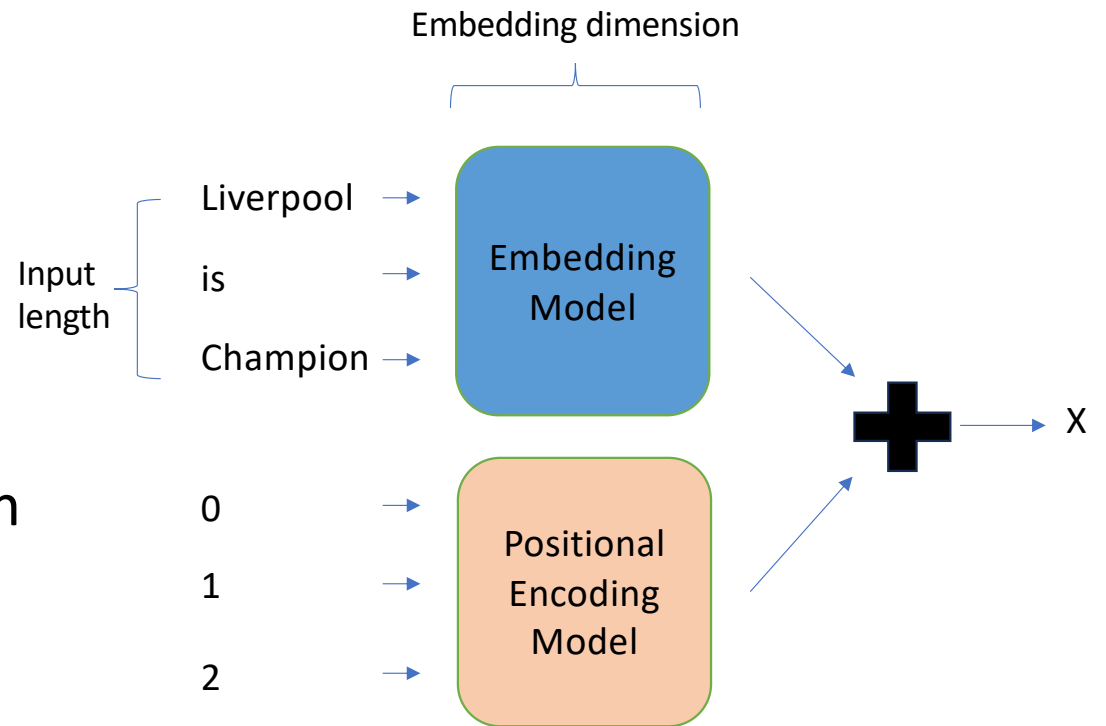
Transformer Unfolded

- English to French translation
- Example on the right:
 - 6 encoder blocks
 - 6 decoder blocks
- Three steps:
 - Input encoding
 - Encoder
 - decoder



Input Encoding

- Encoding $\mathbf{X} = \text{Word Embedding } \mathbf{E} + \text{Positional Encoding } \mathbf{P}$
- $\mathbf{E}$ can be obtained through pre-trained embedding models such as Word2Vec, Glove, etc
- $\mathbf{P}$ represents the position of a word in a sentence.

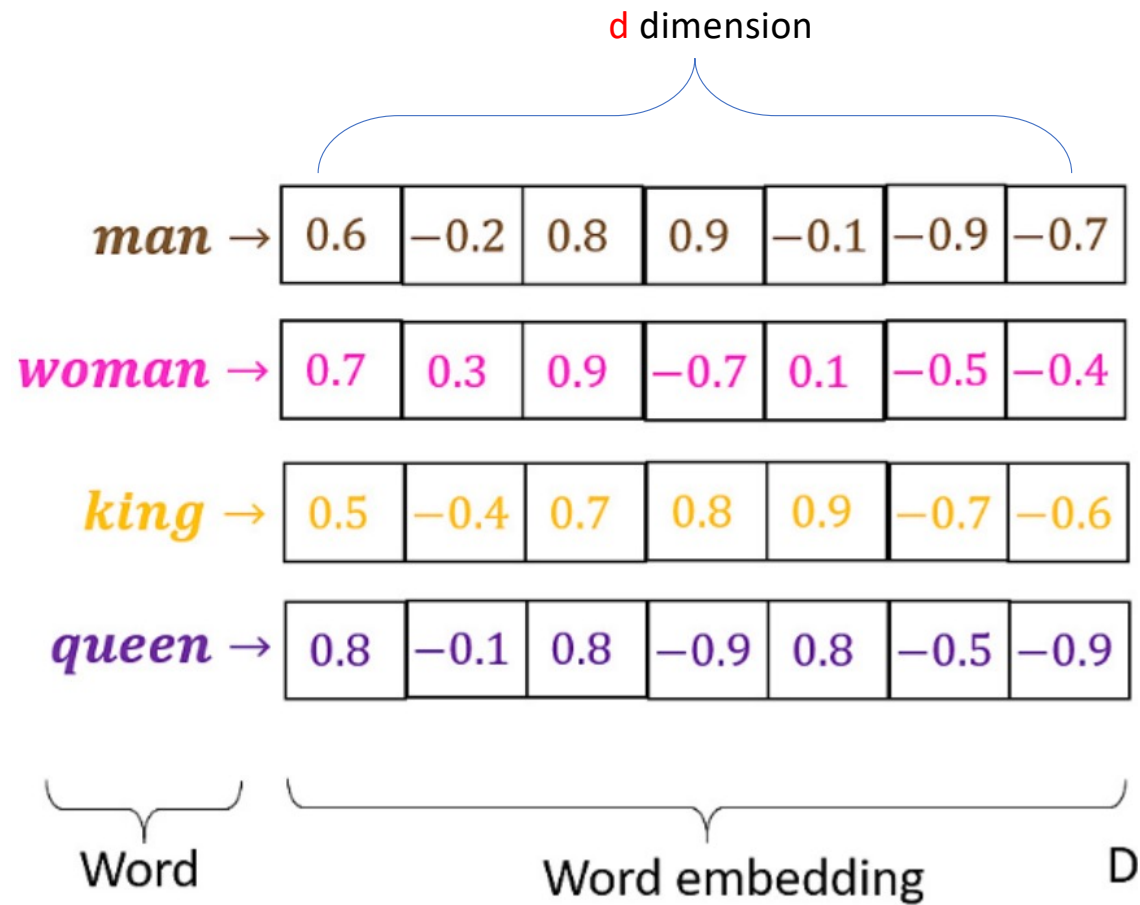


Why $\mathbf{P}$: Transformer needs global information, which is important for NLP. Positional encoding retains the relative or absolute positions of the words in a sentence.

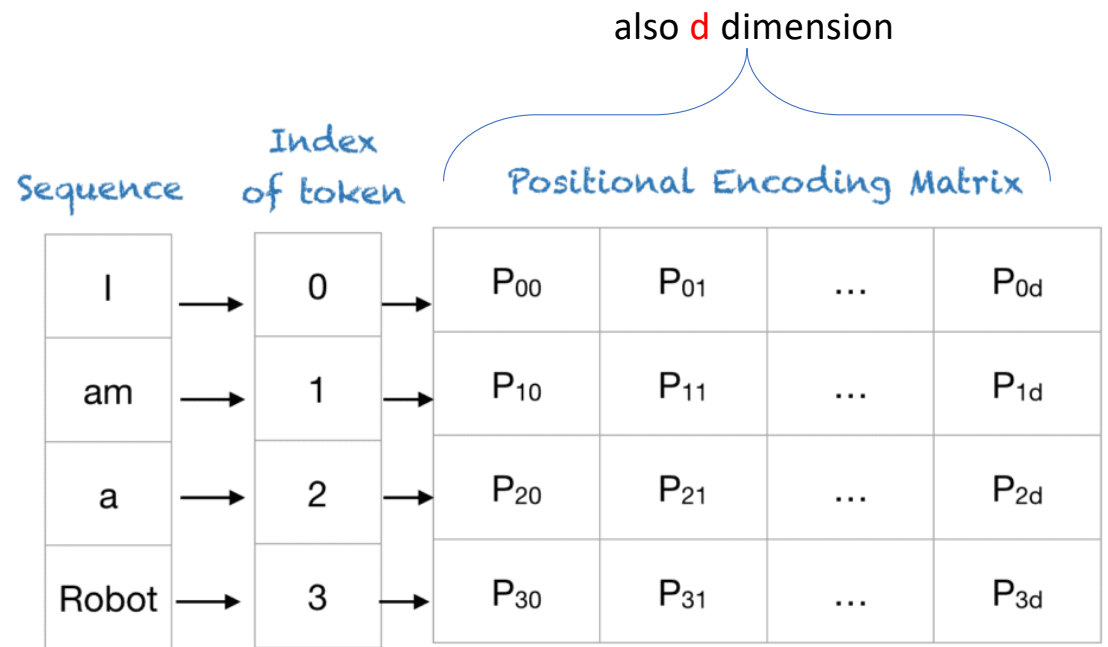
Positional Embedding

- Transformers take in a complete prompt at once (in contrast to RNNs) and embed this as one big Tensor. As such, transformers do not know which word is at what position in the sentence.
- This is problematic as the following two sentences mean entirely different things, only dependent on the order of the words:
 - Liverpool beat Chelsea 2:1
 - Chelsea beat Liverpool 2:1
- To that end, a positional embedding is added to allow the model to deduce which word is where.
- Positional embedding can be learned, or designed (in the original Transformer)

Word Embedding



What is
positional
encoding?



Positional Encoding Matrix for the sequence 'I am a robot'

Then, how to define $P(\text{pos}, d)$, where pos is the index of token and d represents the dimension in the embedding?

Positional Encoding

- P has the same shape with the word embedding. It is possible that P can be obtained through training, but Transformer uses the following formula to compute P directly.

$$\begin{aligned}P_{(pos, 2i)} &= \sin(pos / 10000^{2i/d}) \\P_{(pos, 2i+1)} &= \cos(pos / 10000^{2i/d})\end{aligned}$$

where pos is the position of the word in the sentence, d is the dimension of P, 2i denotes the even dimension, 2i+1 denotes the odd dimension.

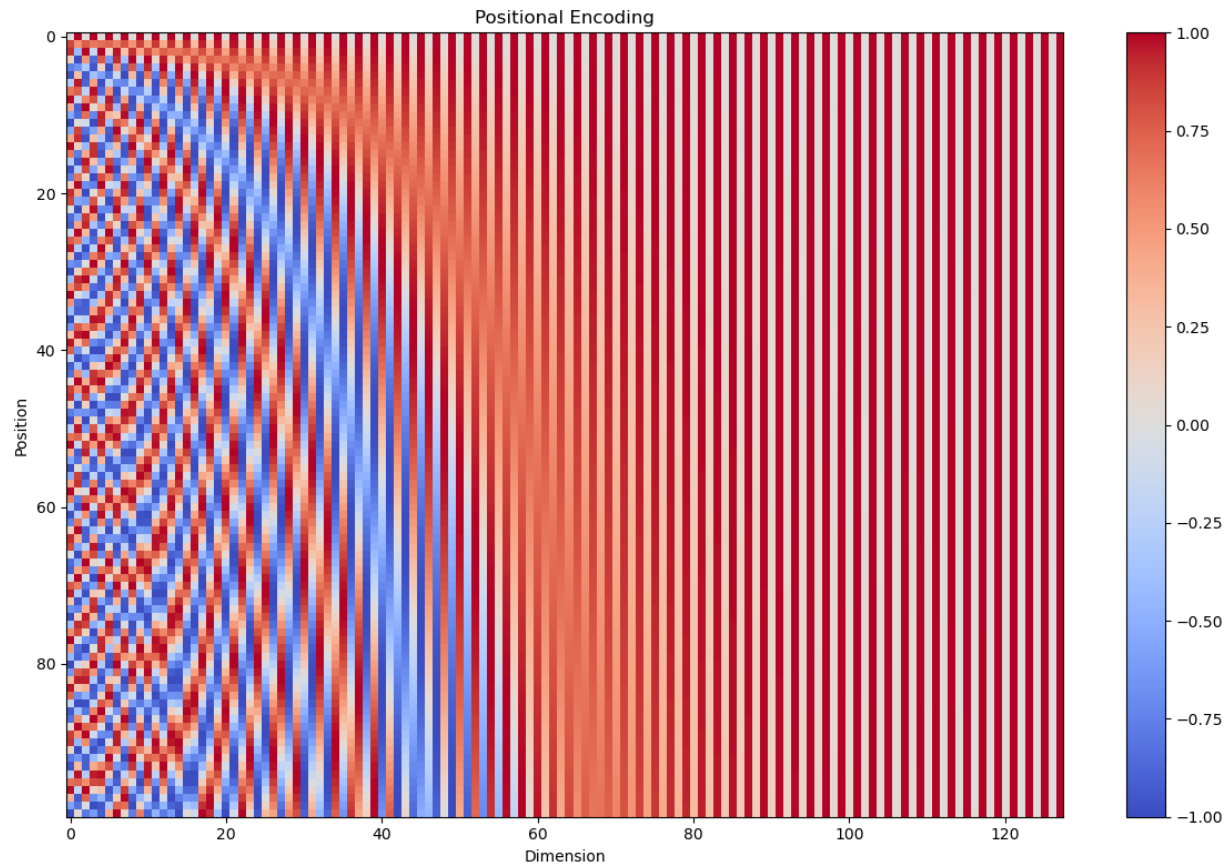
Positional Encoding

P Assuming $d = 4$

	$2i=0$	$2i+1=1$	$2i=2$	$2i+1=3$
pos=0	$P_{00} = \sin(0) = 0$	$P_{01} = \cos(0) = 1$	$P_{02} = \sin(0/10000^{2/4}) = 0$	$P_{03} = \cos(0/10000^{2/4}) = 1$
pos=1	$P_{10} = \sin(1/1) = 0.84$	$P_{11} = \cos(1/1) = 0.54$	$P_{12} = \sin(1/10000^{2/4}) = 0.00999$	$P_{13} = \cos(1/10000^{2/4}) = 0.99995$
pos=2	$P_{20} = \sin(2/1) = 0.91$	$P_{21} = \cos(2/1) = -0.42$	$P_{22} = \sin(2/10000^{2/4}) = 0.01999$	$P_{23} = \cos(2/10000^{2/4}) = 0.99980$

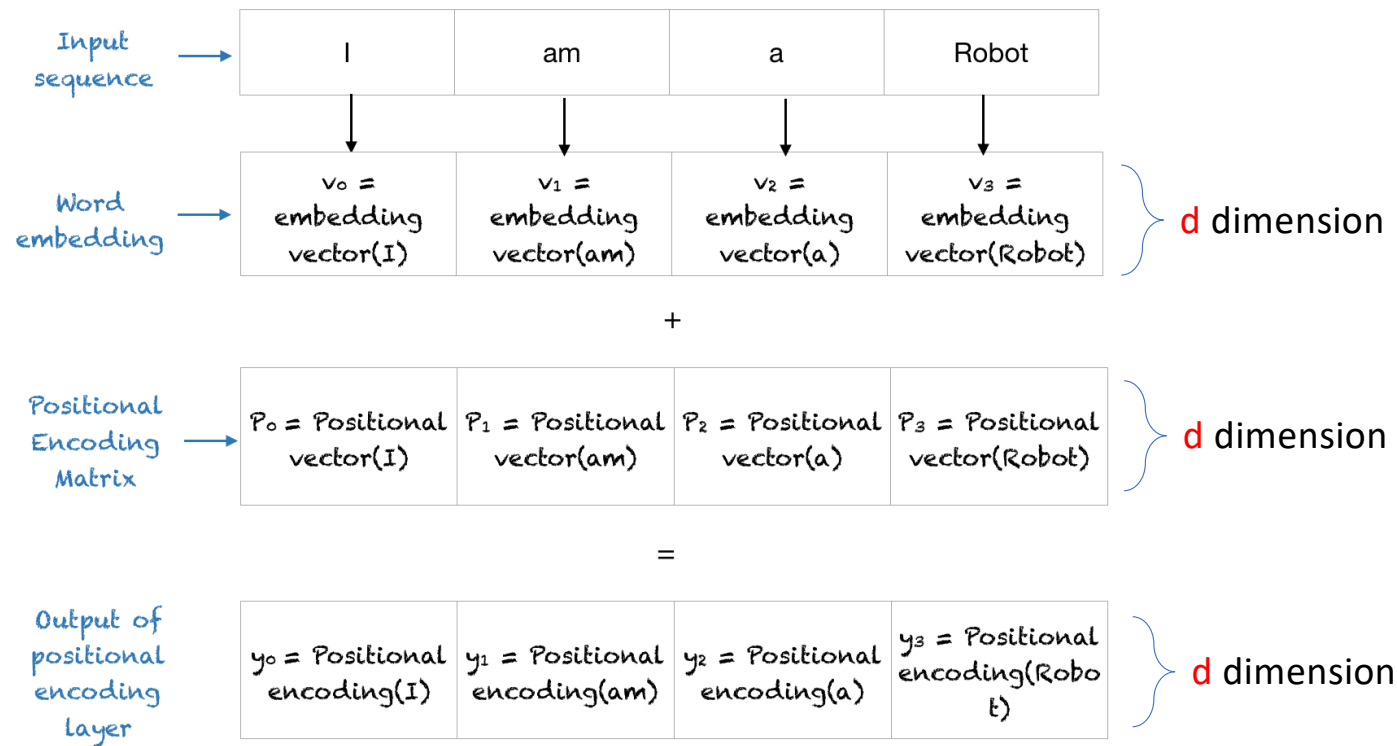
$$\begin{aligned}
 P_{(pos, 2i)} &= \sin(pos/10000^{2i/d}) \\
 P_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d})
 \end{aligned}$$

Visualisation of Positional Encoding



<https://medium.com/thedeephub/positional-encoding-explained-a-deep-dive-into-transformer-pe-65cfe8cfe10b>

Output of Positional Encoding Layer



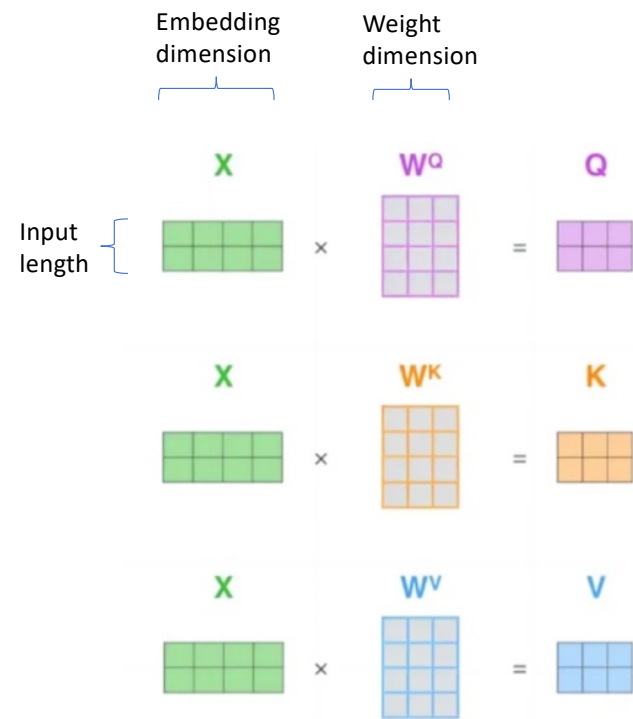
- <https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>

Positional Encoding

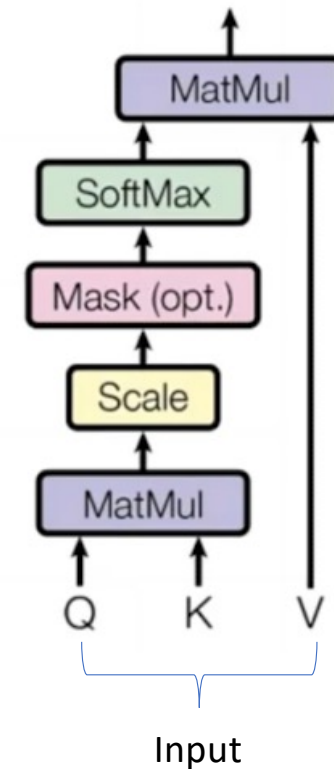
- Can be adapted to sentences longer than those in the training dataset.
 - Assuming that the longest in the training dataset is 100, and we can still use the formulas to compute the embedding for the sentences of length 150
- Easy to compute relative positional encoded, i.e., we can compute $P(\text{pos}+k)$ from $P(\text{pos})$, for a given k , because
 - $\sin(A+B)=\sin(A)\cos(B) + \cos(A)\sin(B)$
 - $\cos(A+B)=\cos(A)\cos(B) - \sin(A)\sin(B)$

Self-Attention

- Q – query
- K – key
- V – value
- W^Q , W^K , W^V are linear transformation matrices



Scaled Dot-Product Attention



Self-Attention

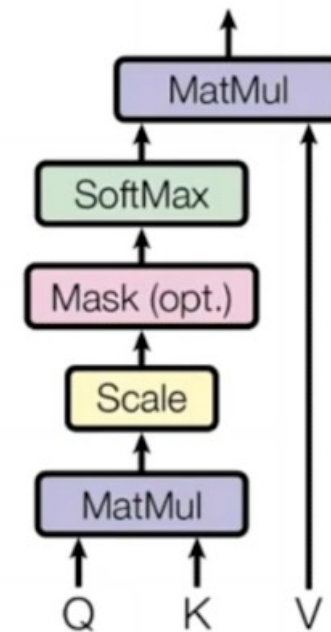
This is where the computational power is heavily used.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

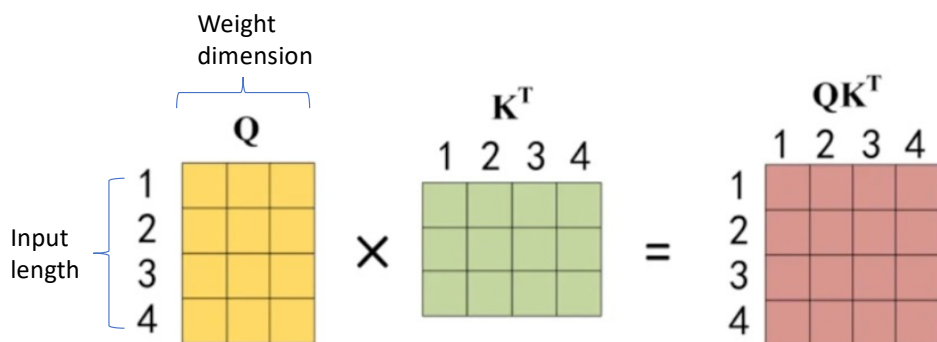
where d_k is the number of columns in Q and K , that is, the dimension of vectors.

Let's break it down!

Scaled Dot-Product Attention



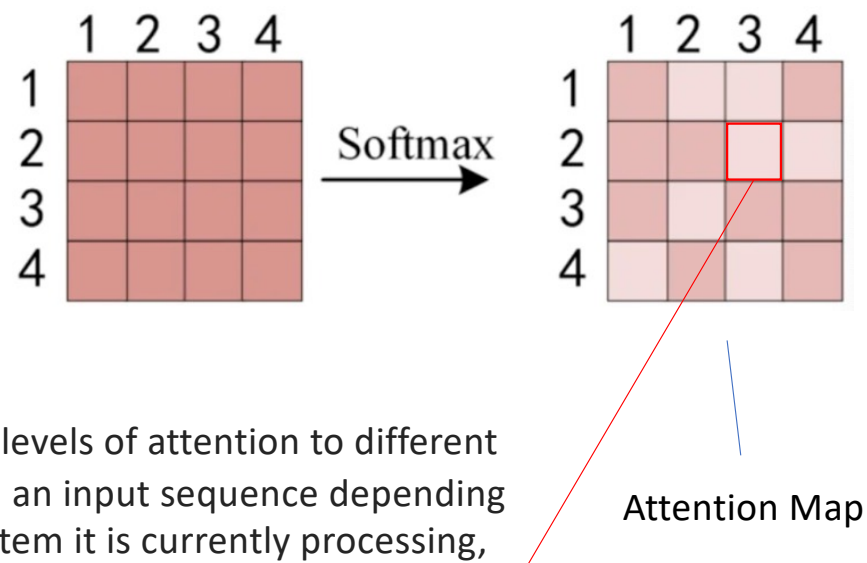
QK Attention Map



- $\text{Sqrt}(d_k)$ is to ensure that QK^T will not be too big.
- The obtained matrix has n columns and rows, where n is equivalent to the number of words in the input.

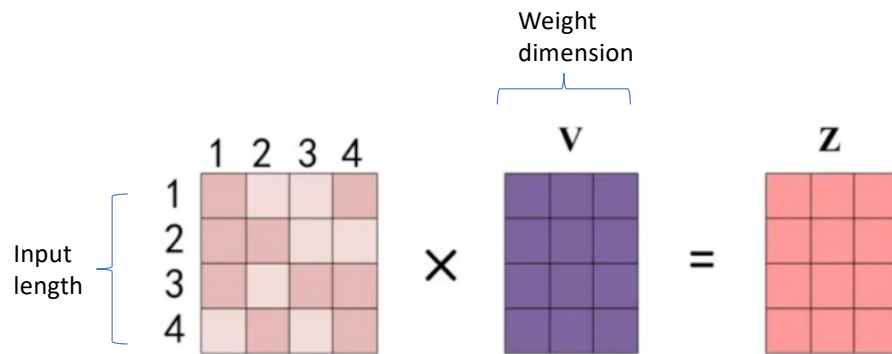
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Softmax: over every row, to make it into probability.



varying levels of attention to different items in an input sequence depending on the item it is currently processing, e.g., the attention level of the third token when processing the second token.

Self-Attention



- Every row of the softmax matrix is a probability distribution.

Diagram illustrating the calculation of the first row of the output Z , z_1 :

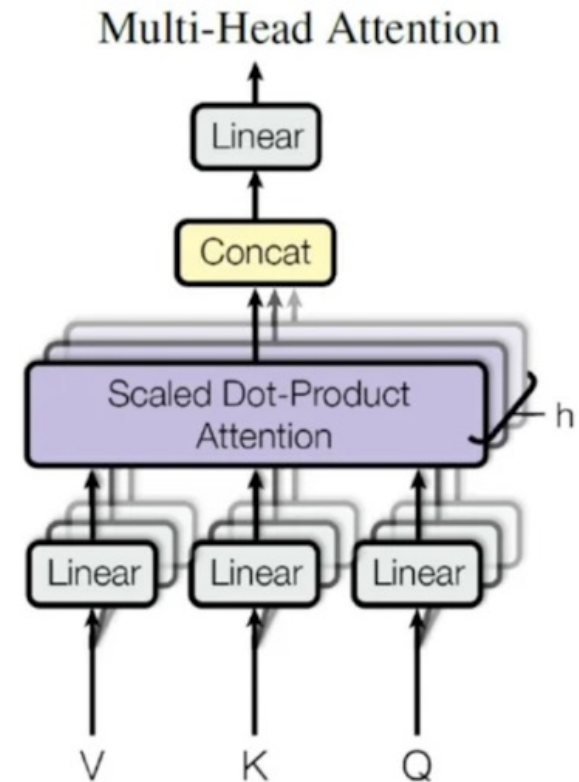
$$z_1 = 1 \begin{bmatrix} 0.3 & 0.2 & 0.2 & 0.3 \end{bmatrix} \times V$$

$$= 0.3 \times \text{row 1 of } V + 0.2 \times \text{row 2 of } V + 0.2 \times \text{row 3 of } V + 0.3 \times \text{row 4 of } V$$

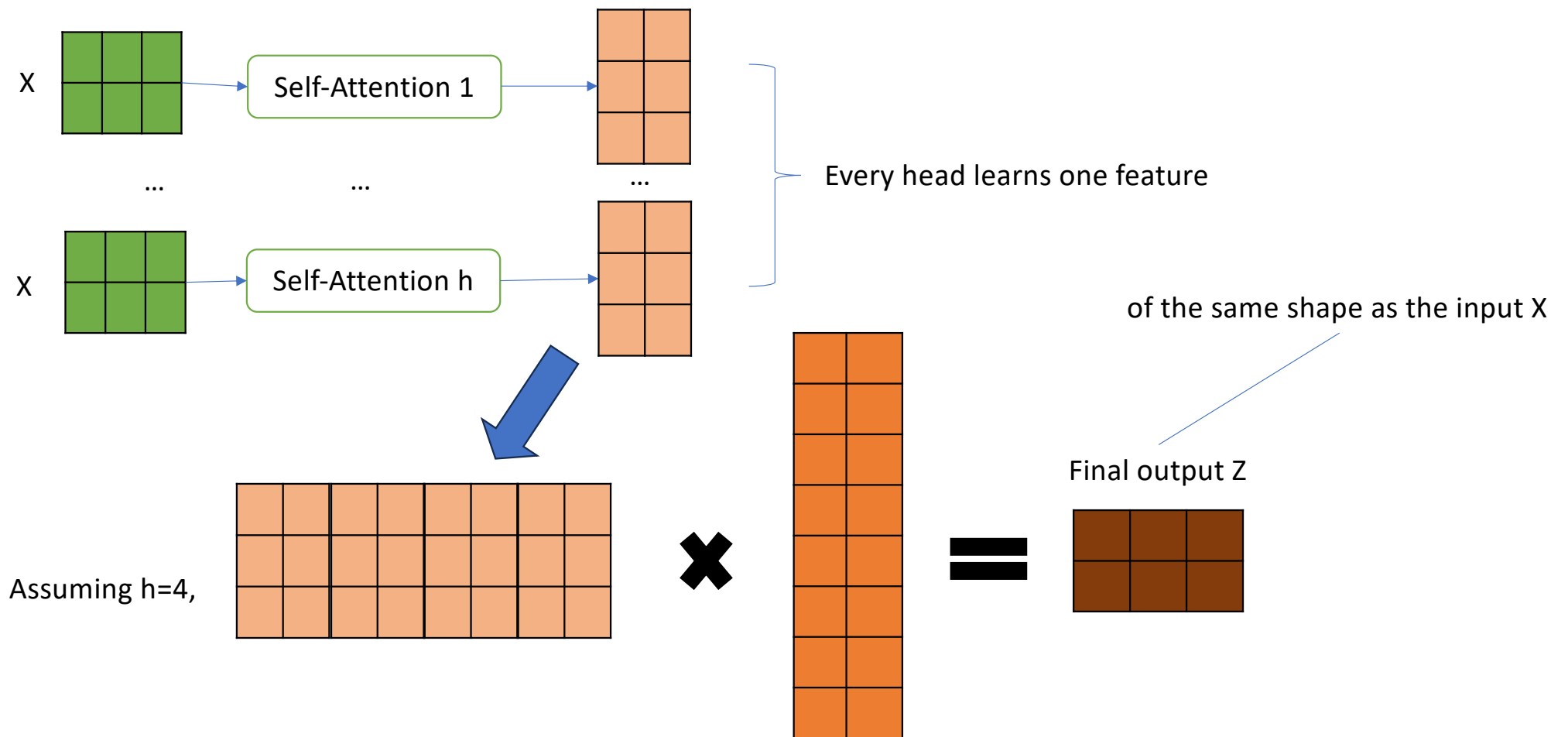
- Every Z_i is a weighted value vector, for a row of the softmax matrix

Multi-Head Attention

- Formed of multiple self-attention, **to learn multiple lower-scale feature maps**
- X was passed to h self-attention, to get h different Z s
- Z s are concatenated, before conducting a linear transformation to get the final output Z .



Multi-Head Attention

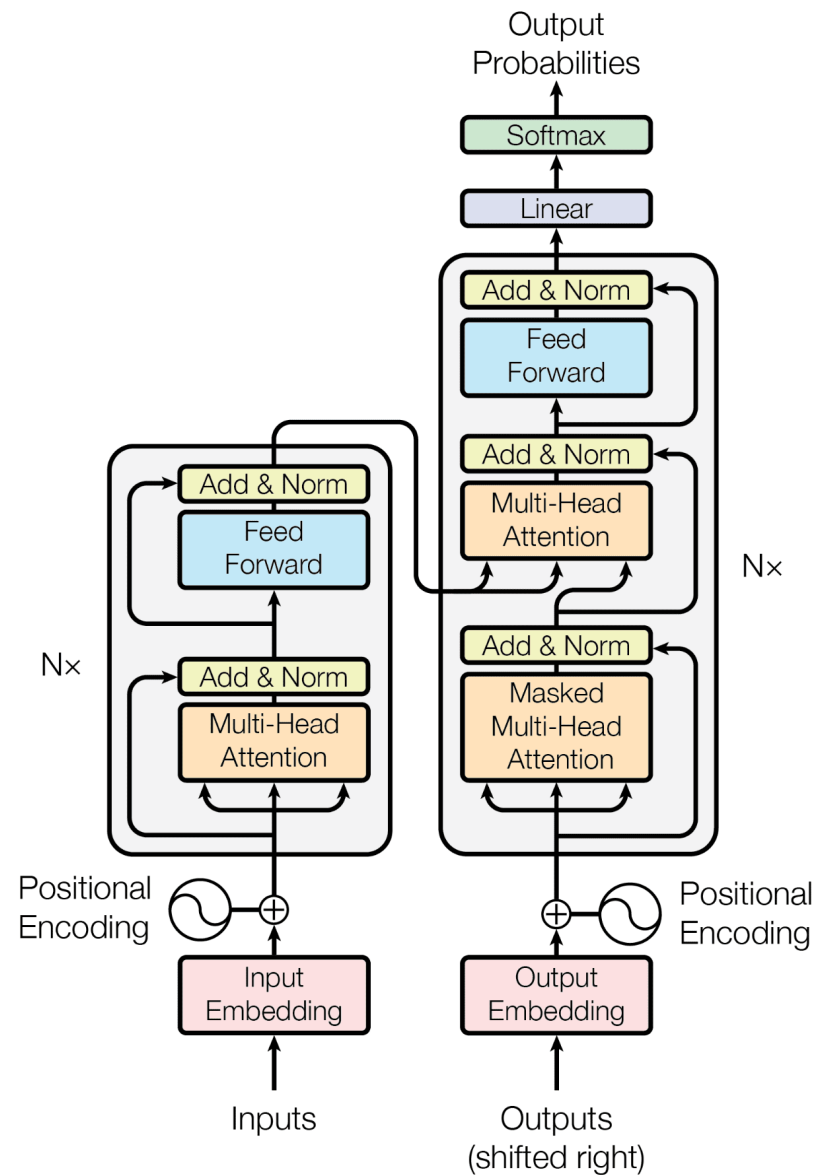


Add & Norm

$\text{LayerNorm}(X + \text{MultiHeadAttention}(X))$

$\text{LayerNorm}(X + \text{FeedForward}(X))$

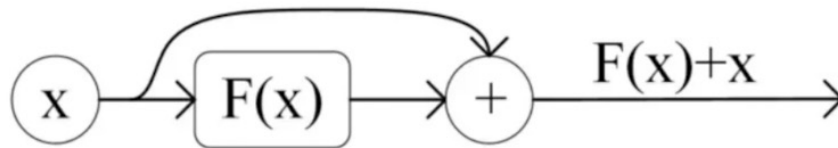
- X is the input for either MultiHeadAttention or FeedForward
- MultiHeadAttention(X) and FeedForward(X) have the same dimension as X.



Add & Norm

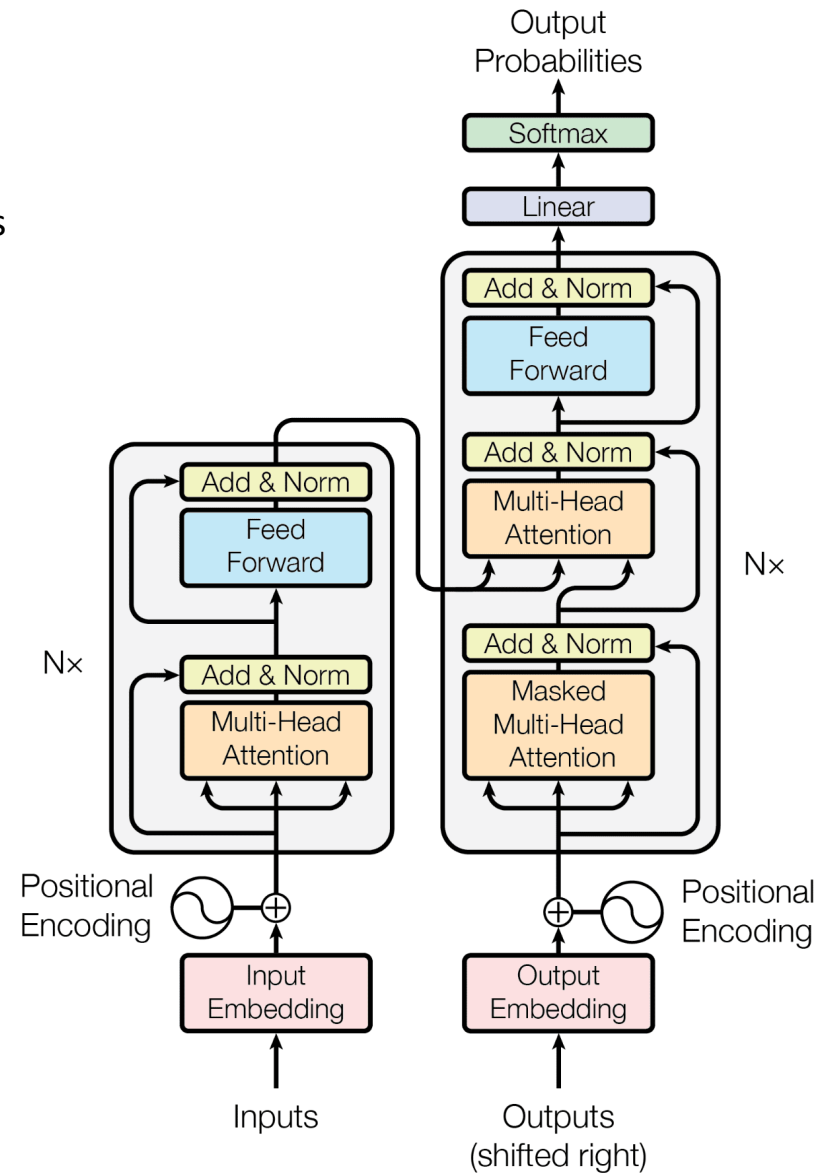
Residual Layer as in CNNs

- **Add** refers to the + operation, can be seen as a residual connection:



- **Norm** refers to layer normalisation, converting activations of all layers to have the same mean and variance.

Similar as Batchnorm layer in CNNs

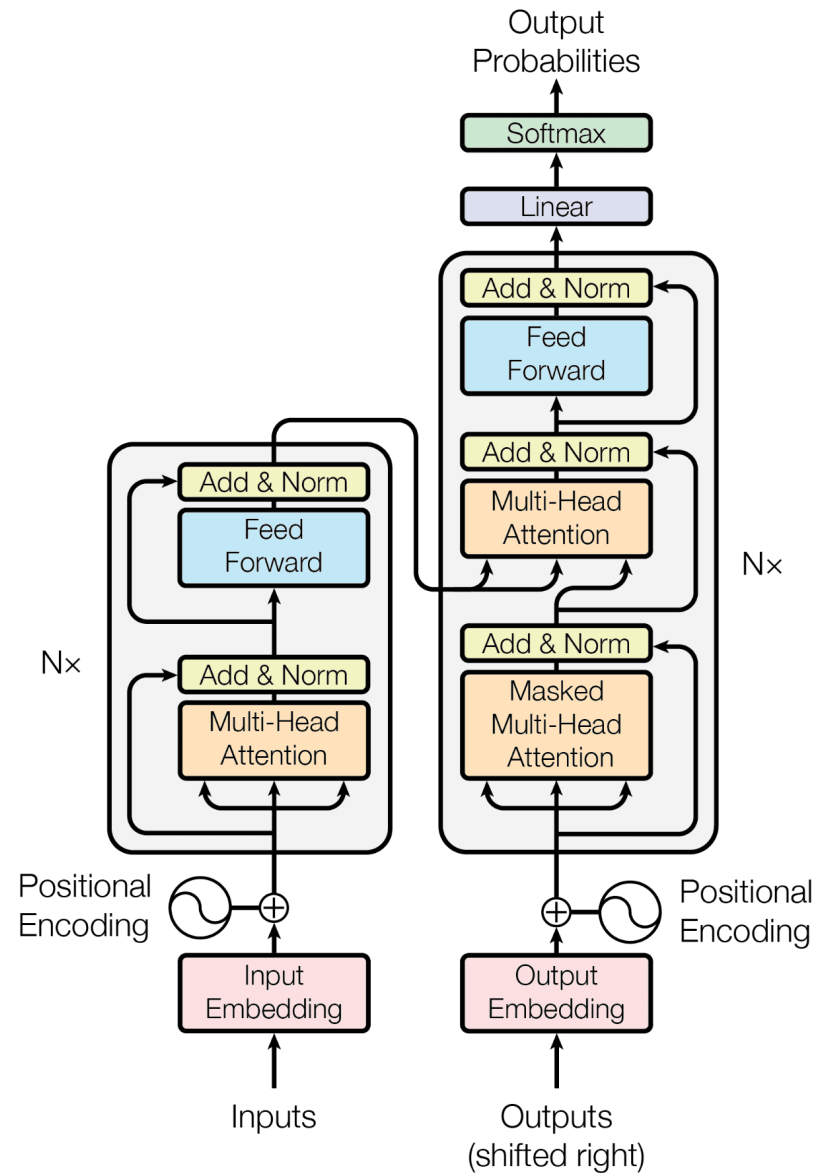


Feedforward

- A two-layer MLP, with the first layer having the ReLU activation function and the second layer having no activation function:

$$\max(0, XW_1 + b_1)W_2 + b_2$$

- Output has the same shape with X.

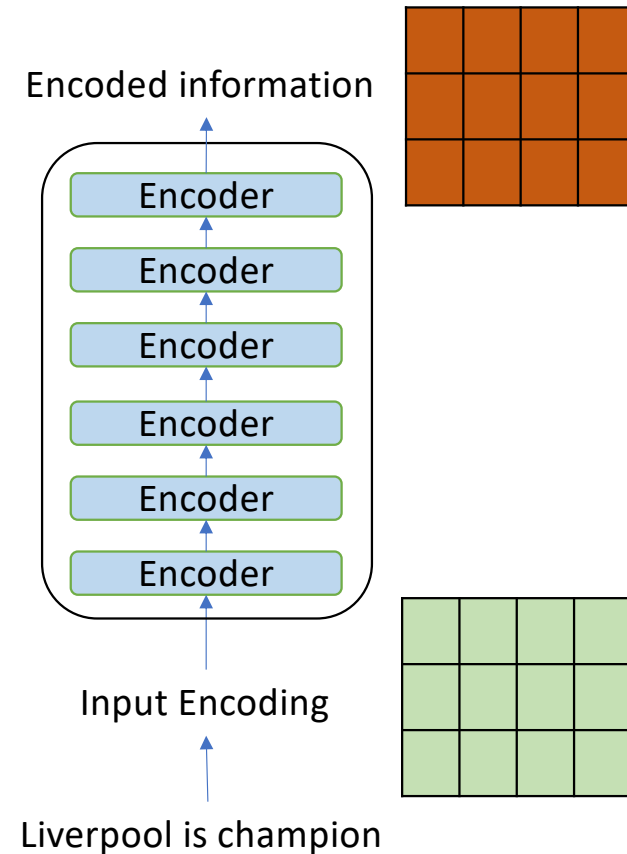


Encoder

Nx encoder blocks.

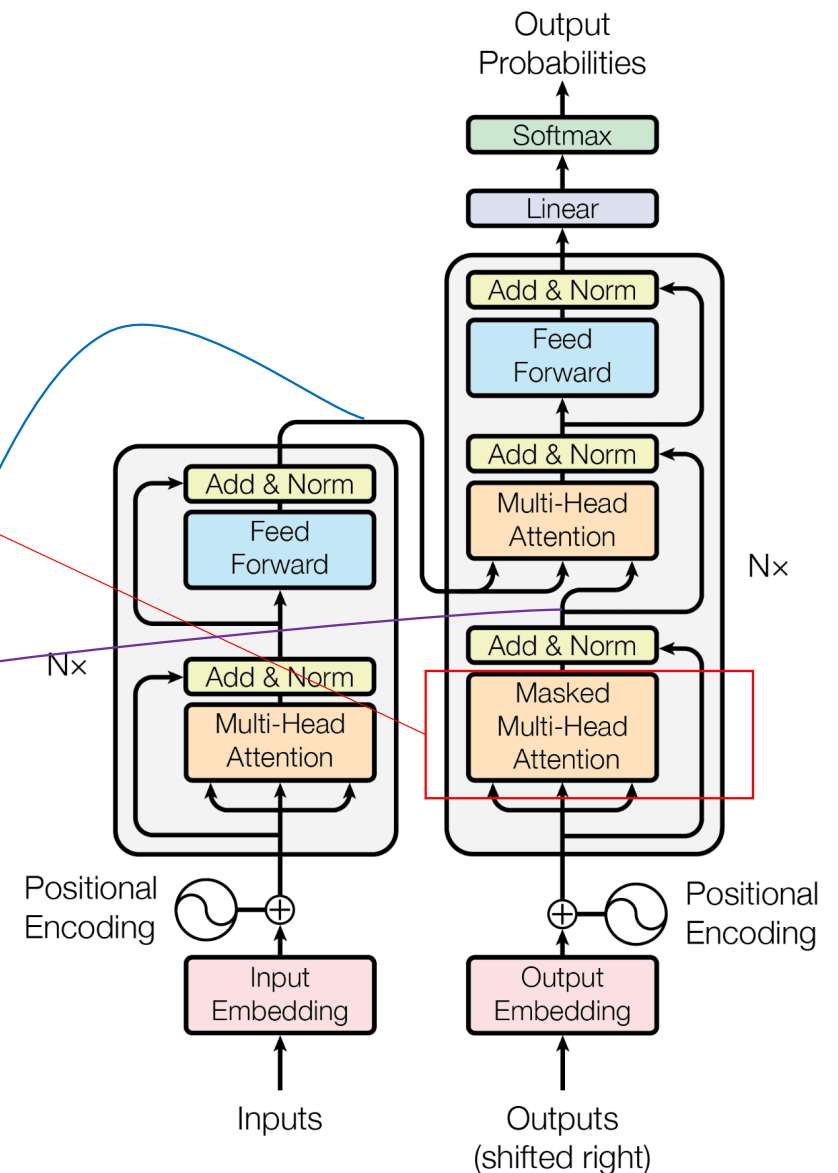
The first encoder has the input encoding as the input;

The later encoder has the output of the previous encoder as the input;

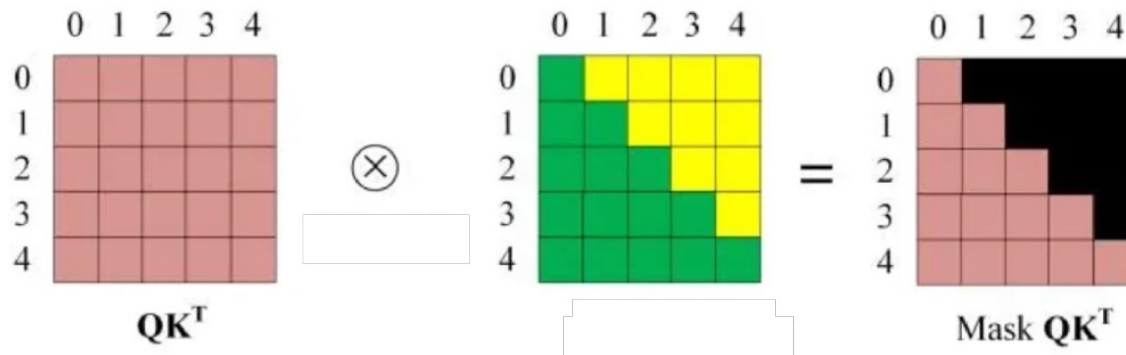


Decoder

- Two multi-head attention layers.
- The first layer uses **masking operation**.
- In the second layer,
 - K and V use the output of the encoder as the input, and
 - Q use the output of the first decoder layer as the input.
- The final softmax computes the **probability for the next word**.



Mask

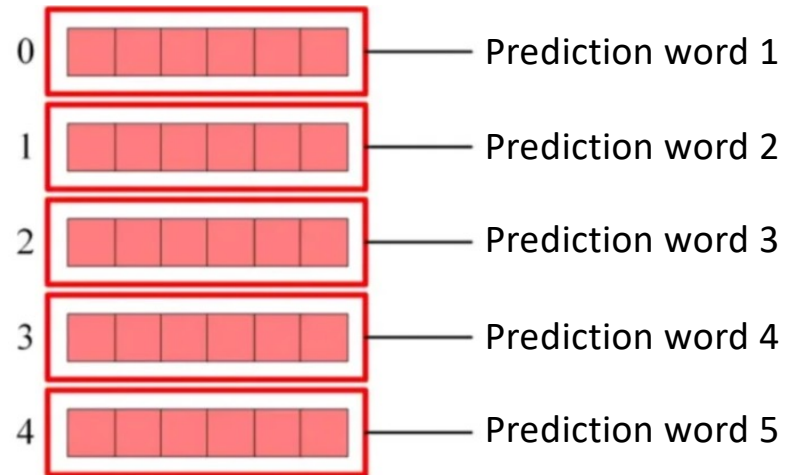


where **green** represents unmasked (1) and **yellow** represents masked (0).

- The masking operation is conducted before applying softmax
- The 'masking' term ensures that the encoder could see the whole (original language) sentence, and the decoder could only see the first part of the sentence which was already translated.

Outputs: predicting next words

- Masking enables
 - Z0 only has information about the first word in the input
 - Z1 only has information about the first and second words in the input,
 - Etc



Transformer Summary:

- Unlike RNN, it can be more efficiently trained
- Positional encoding enables its awareness of the positions of the words
- Self-attention is the core methods, using Q, K, V matrices and linear transformations.
- Multihead attention can captures multiple attention scores